Atlas Environment Configuration Report

2.5 Environment Variables & Dotfile Customisation • Atlas Developer Environment Verification

STUDENT NAME	SECTION / BATCH	STUDENT ACCOUNT	DATE & PLATFORM
Vivegan K	**Kalvium CSE (Batch 2026)**	**vivegan_ko (vivegan-21-7)**	**Sep 16, 2026 • macOS zsh**

Mission Objective: Configure environment variables, modify the system PATH safely, persist configuration and aliases in dotfiles (`~/.bashrc` / `~/.zshrc`), and establish a secure `.env` configuration vault isolated with `.gitignore`. All five missions have been executed directly on the native macOS Darwin terminal.

Mission 1 — Environment Scanner

Executed Commands: `cd ~/Project-Atlas`, `pwd`, `echo $HOME`, `echo $USER`, `echo $PATH`, `env | head`

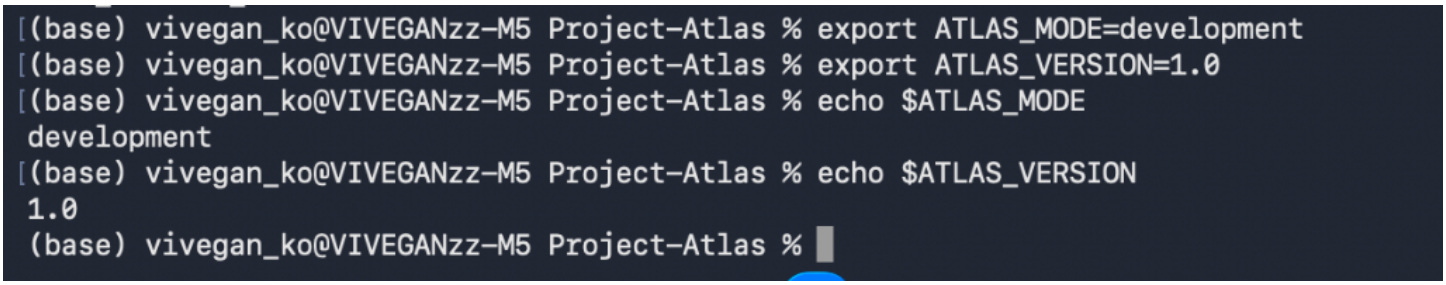


Screenshot 1 — Environment Scanner (Verified Execution & Output)

Short Explanation:
An **environment variable** is a dynamic key-value pair maintained by the operating system shell that provides configuration settings, user identity, and runtime context to running programs and subprocesses. Specifically, **$HOME** represents the absolute path to the active user's personal directory (`/Users/vivegan_ko`), whereas **$PATH** defines the ordered list of filesystem directories the shell systematically searches to discover and run executable command binaries.

Mission 2 — Configuration Builder

Executed Commands: `export ATLAS_MODE=development`, `export ATLAS_VERSION=1.0`, `echo $ATLAS_MODE`, `echo $ATLAS_VERSION`

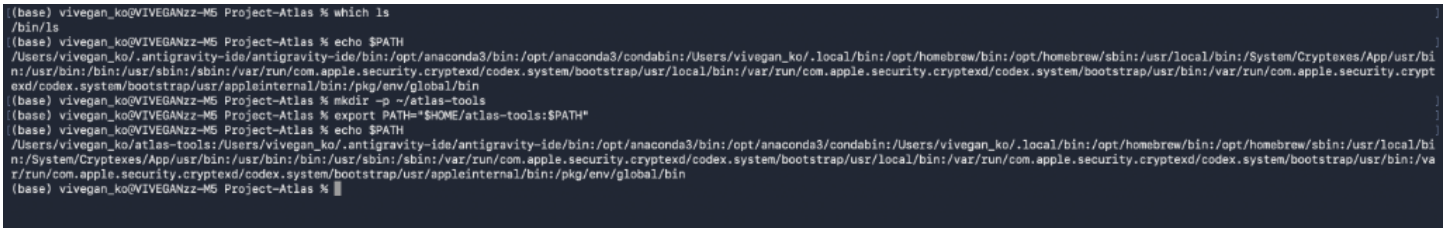


Screenshot 2 — Configuration Builder (Verified Execution & Output)

Short Explanation:
The **export** command defines a variable and exports it into the shell environment so that it is inherited by any subsequent programs and child subshells launched from that terminal session. Applications read settings from environment variables so that configuration (such as database URLs, feature flags, and deployment stages) remains completely decoupled from source code, allowing identical code to execute across different environments.

Mission 3 — PATH Navigator

Executed Commands: `which ls`, `echo $PATH`, `mkdir -p ~/atlas-tools`, `export PATH="$HOME/atlas-tools:$PATH"`, `echo $PATH`



Screenshot 3 — PATH Navigator (Verified Execution & Output)

Short Explanation:
The **PATH** variable tells the shell which filesystem directories to search in order from left to right whenever a command is typed, automatically invoking the first executable match found. Prepending our custom folder using `:$PATH` is safe because it preserves all existing system search paths (like `/bin` and `/usr/bin`); replacing PATH outright would break essential shell commands such as `ls` and `cd`.

Mission 4 — Shell Architect

Executed Commands: `echo $SHELL`, `nano ~/.bashrc`, `source ~/.bashrc`, `atlas`, `pwd`, `ll`

```
── 1. Aliases Configured in ~/.bashrc ──

alias atlas="cd ~/Project-Atlas"
alias ll="ls -lah"

── 2. Terminal: Reloading Shell & Testing Aliases (source, atlas, pwd, ll) ──
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % echo $SHELL
/bin/zsh
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % nano ~/.bashrc
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % source ~/.bashrc
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % atlas
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % pwd
/Users/vivegan_ko/Project-Atlas
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % ll
total 0
drwxr-xr-x   8 vivegan_ko  staff   256B  1 Sep 10:59 .
drwxr-x---+ 50 vivegan_ko  staff   1.6K 16 Sep 08:56 ..
drwxr-xr-x   2 vivegan_ko  staff    64B  1 Sep 10:57 assets
drwxr-xr-x   6 vivegan_ko  staff   192B  1 Sep 10:58 backend
drwxr-xr-x   3 vivegan_ko  staff    96B  1 Sep 10:58 docs
drwxr-xr-x   5 vivegan_ko  staff   160B  9 Sep 08:33 frontend
-rw-r--r--   1 vivegan_ko  staff     0B  1 Sep 10:59 README_BACKUP.md
-rw-r--r--   1 vivegan_ko  staff     0B  1 Sep 10:57 README_PROJECT.md
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas %
```

Screenshot 4 — Shell Architect (Dotfile Configuration & Verification)

Short Explanation:
Aliases provide custom shorthand shortcuts for repetitive or lengthy commands (such as navigating directly to Project Atlas or viewing detailed directory listings with `ll`), significantly boosting developer productivity and minimizing typing mistakes. The **source** command reads and executes the commands from a specified dotfile directly within the current interactive shell session, immediately loading newly declared aliases and settings without requiring the user to close and reopen the terminal.

Mission 5 — Atlas Configuration Vault

Executed Commands: `cat .env`, `cat .gitignore`, `export ATLAS_APP_NAME=ProjectAtlas`, `export ATLAS_ENV=development`, `export ATLAS_API_URL=http://localhost:3000`, `node config-demo.js`

```
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % cat .env
ATLAS_APP_NAME=ProjectAtlas
ATLAS_ENV=development
ATLAS_API_URL=http://localhost:3000
ATLAS_API_KEY=atlas-demo-key-123
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % cat .gitignore
.env
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % export ATLAS_APP_NAME=ProjectAtlas
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % export ATLAS_ENV=development
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % export ATLAS_API_URL=http://localhost:3000
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas % node config-demo.js
Application: ProjectAtlas
Environment: development
API URL: http://localhost:3000
(base) vivegan_ko@VIVEGANzz-M5 Project-Atlas %
```

Screenshot 5 — Atlas Configuration Vault (Protected Config & Execution)

Short Explanation:
Configuration settings and sensitive secrets must live outside application source code to prevent accidental credential leakage and ensure that the exact same codebase can deploy seamlessly to development, testing, and production servers without code modifications. The **.env** file is explicitly listed in **.gitignore** so that Git ignores the file entirely, guaranteeing that environment-specific secrets, credentials, and API keys are never tracked, committed, or pushed to remote repositories.

Engineering Takeaway & Security Best Practices

The Golden Rule of Twelve-Factor Applications: *Configuration belongs outside code; secrets definitely do not belong in code.* Through these five missions, Project Atlas transitions from a rigid hardcoded structure to a resilient, environment-driven architecture. Dotfiles (`~/.bashrc` / `~/.zshrc`) establish a productive local developer environment, while project vaults (`.env`) coupled with `.gitignore` safeguard sensitive keys. At runtime, Node.js retrieves dynamic values seamlessly through `process.env`, achieving total environment portability.

Mission	Core Concept	Operational Mechanism	Status
Mission 1	Inspect Variables	System inspect with `echo` & `env | head`	**PASSED**
Mission 2	Create Variables	Export session variables via `export`	**PASSED**
Mission 3	PATH Navigation	Safe folder prepending with `:$PATH`	**PASSED**
Mission 4	Dotfile Persistence	Shortcuts in `~/.bashrc` reloaded with `source`	**PASSED**
Mission 5	Configuration Vault	Protected `.env`, ignored via `.gitignore`, read by Node	**PASSED**